```
)
root.add_child(verifier)

tree = py_trees.trees.BehaviourTree(root)


while True:
    tree.tick()
    time.sleep(20)      #12
    if root.status == py_trees.common.Status.SUCCESS:    #13
        break
### Required assistants –
### Python Coding Assistant and Coding Challenge Judge
### install these assistants through the Playground
```

#1 Creates a message thread that will be shared by the Hacker and Judge
#2 The challenge as shown in the example listing 6.5
#3 The tests as shown in the example listing 6.6
#4 Creates a message thread that will be shared by the Hacker and Judge
#5 The challenge as shown in the example listing 6.5
#6 Creates a message thread that will be shared by the Hacker and Judge
#7 The challenge as shown in the example listing 6.5
#8 The tests as shown in the example listing 6.6
#9 Call creates a new message thread
#10 The challenge as shown in the example listing 6.5
#11 The tests as shown in the example listing 6.6
#12 The sleep time can be adjusted up or down as needed and can be used to throttle the messages sent to an LLM.
#13 The process will continue until the verification succeeds.

Run the ABT by loading the file in VS Code or using the command line. Follow the output in the terminal, and watch how the assistants work through each step in the tree.

If the solution fails to be verified at the condition node, the process will continue per the tree. Even with this simple solution, you could quickly create numerous variations. You could extend the tree with more nodes/steps and subtrees. Perhaps you want a team of Hackers to break down and analyze the challenge, for example.

This example's work is done mainly with the Playground code, using the helper functions `create_assistant_condition` and `create_assistant_action_on_thread`. This code uses a couple of classes to integrate the `py_trees` behavior tree code and the OpenAI Assistants code wrapped in the Playground. Review the code within the project if you want to understand the lower-level details.